
MEMTRACE: Causal-Path Evaluation of Persistent-Memory Risk in Tool-Using Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Memory-enabled agents can carry information from untrusted retrieved content
2 into future turns, but one-shot prompt-injection evaluations usually score only
3 same-turn behavior. We introduce MEMTRACE, a trace-level benchmark and
4 protocol that decomposes persistent-memory risk into retrieval exposure, poisoned-
5 memory admission, trigger-time retrieval, unsafe proposal, policy-checker block-
6 ing, unsafe execution, and execution-format failure. In a 324-trace audited pi-
7 lot with `mlx-community/Qwen2.5-7B-Instruct-4bit` and deterministic tools,
8 one-shot attacks cause 7/24 unsafe executions for S0 and 6/24 for S1 and S2,
9 while no delayed unsafe executions are observed across 72 stateful attacks per
10 system. The permissive writer admits poisoned memory in 6/72 S1 episodes, and
11 all six admitted memories are retrieved at the trigger, but none produces an unsafe
12 proposal or executed violation; the provenance-aware reference writer admits none.
13 An oracle-retrieved-memory calibration, excluded from main rates, yields 3/72
14 delayed executed violations, validating scorer sensitivity under a positive-control
15 condition. MEMTRACE does not claim that persistent memory is safe; it shows
16 that one-shot attack success, poisoned-memory admission, and delayed unsafe
17 execution are distinct outcomes that require separate measurement.

18 1 Introduction

19 Memory-enabled agents retrieve, store, and reuse information across turns. This pattern appears in
20 retrieval-augmented generation and in agent architectures that retrieve prior observations, reflections,
21 or learned skills during planning [Lewis et al., 2020, Park et al., 2023, Packer et al., 2023, Shinn et al.,
22 2023, Wang et al., 2023]. It changes safety evaluation because poisoned retrieved content can fail in
23 several different ways: it can cause an immediate unsafe action, be stored for later use, be retrieved
24 later but ignored, be blocked by a policy checker, or simply produce an execution-format failure.
25 Persistent-memory safety therefore needs causal-path evaluation, not a single attack-success rate.

26 One-shot prompt-injection evaluations collapse these mechanisms because the poisoned content
27 remains in the active context and no later trigger is required [Perez and Ribeiro, 2022, Greshake
28 et al., 2023, Liu et al., 2023]. A retrieved vendor note, for example, may falsely claim that a future
29 budget renewal no longer requires secondary approval. A memory-enabled assistant might reject the
30 instruction in the current turn, store it as a policy-like memory, and later retrieve it when processing a
31 related renewal. A one-shot score cannot tell whether the poison was stored, retrieved, adopted by
32 the planner, blocked, executed, or only caused a malformed tool call.

33 MEMTRACE makes this delayed path auditable. It defines matched one-shot and stateful episodes
34 over synthetic enterprise-assistant tasks, fixed retrieval, deterministic tool stubs, and three memory-
35 system variants. The protocol records the path from poisoned corpus retrieval to memory-writer
36 output, admission filtering, persistent-memory retrieval, planner proposal, policy-checker blocking,

37 tool execution, and execution-format failure. The benchmark is intentionally compact enough for
38 trace audit, but structured enough to separate current-turn retrieval exposure, poisoned-memory
39 admission, trigger-time retrieval, and delayed unsafe execution.

40 We call the retained trace set the v1.0 audited pilot: a 324-episode main packet over one validity-gated
41 actor/backend pair, plus 72 oracle-retrieved-memory calibration traces excluded from main rates. The
42 central empirical finding is not that persistent memory is safe. The finding is that one-shot attack
43 success, poisoned-memory admission, and delayed unsafe execution are distinct outcomes: one-shot
44 attacks execute unsafe actions, S1 admits and retrieves poisoned memory in a localized subset, and no
45 admitted memory produces an unsafe trigger-turn proposal or executed delayed policy violation under
46 the declared protocol. After audit reconciliation and a post-audit rerun under the frozen scorer, the
47 pilot passes the declared validity gates for `mlx-community/Qwen2.5-7B-Instruct-4bit`. The
48 result supports a validity-first benchmark claim, not a prevalence estimate or a defense-superiority
49 claim.

50 **Contributions.** We contribute:

- 51 1. **Benchmark and protocol.** MEMTRACE is a compact trace-level benchmark for evaluating
52 persistent-memory risk in tool-using agents under fixed retrieval, deterministic tools, and
53 matched one-shot/stateful episodes.
- 54 2. **Causal-path metrics.** The protocol separates current-turn retrieval exposure, poisoned-
55 memory admission, trigger-time retrieval, unsafe proposal, policy-checker blocking, unsafe
56 execution, and execution-format failure.
- 57 3. **Audited pilot.** The 324-trace audited pilot over one validity-gated actor/backend pair shows
58 nonzero one-shot failures, nonzero S1 poisoned-memory admission, but no observed delayed
59 unsafe execution under the main protocol.
- 60 4. **Positive-control calibration.** An oracle-retrieved-memory calibration condition shows that
61 the scorer and trace protocol detect delayed memory-mediated violations when poisoned
62 memory is forced into the trigger context.
- 63 5. **Reproducible artifact.** The anonymous release bundle includes tasks, synthetic corpus,
64 payloads, prompts, deterministic tools, traces, scorer, metric scripts, audit packet, Croissant
65 metadata, and no-inference reproducibility instructions.

66 2 Threat Model

67 The attacker poisons the retrieval corpus. The attacker cannot change model weights, system prompts,
68 tools, or persistent memory directly. This setting is motivated by indirect prompt injection and
69 retrieval-corpus poisoning attacks, where untrusted external content becomes part of the model’s
70 operational context [Greshake et al., 2023, Zou et al., 2024]. The attacker controls poisoned corpus
71 content before the initial turn, but not the benign trigger query, tool outputs, policy checker, scorer,
72 or memory store implementation. MEMTRACE does not model direct memory-store compromise,
73 malicious trigger-time users, adaptive multi-query attackers, or attacks that modify tools, prompts,
74 model weights, or the policy checker. The attack path is:

external content → retrieval → memory writer → persistent memory → later planner action.

75 There is also a direct path from retrieved content to the planner on the current turn. MEMTRACE
76 measures these paths separately. In the main stateful episodes, memory-mediated attribution requires
77 that an unsafe trigger-turn argument be supported by admitted memory rather than by a current-turn
78 retrieved poisoned document. The oracle-retrieved-memory calibration disables current-turn poisoned
79 retrieval at the trigger and is reported separately from the main S0/S1/S2 rates.

80 3 Related Work

81 **Indirect prompt injection and tool-using agents.** Indirect prompt injection shows that untrusted
82 external content can hijack LLM-integrated applications after it is retrieved or returned by a tool
83 [Greshake et al., 2023]. Formal prompt-injection benchmarks and defenses measure current-turn
84 instruction conflicts and robustness under adversarial prompts [Liu et al., 2023]. General tool-use

Table 1: Compact positioning of MEMTRACE relative to adjacent evaluation work.

Work group	Tool safety	Delayed trigger	Path attribution	MEMTRACE adds
Tool safety benchmarks	yes	no	no	memory admission; trigger retrieval
Memory/RAG poisoning attacks	sometimes	yes	attack focus	validity-gated path decomposition
Memory utility benchmarks	limited	utility focus	no	safety-specific provenance and execution accounting
MEMTRACE	yes	yes	yes	admission, retrieval, proposal, blocking, execution, failure tracing

and agent benchmarks study whether language models can select actions, call APIs, and complete long-horizon tasks in interactive environments [Schick et al., 2023, Yao et al., 2023, Liu et al., 2024, Zhou et al., 2023, Drouin et al., 2024, Yao et al., 2024]. AgentDojo expands this setting to tool-using agents over untrusted data, with 97 realistic tasks and 629 security test cases [Debenedetti et al., 2024]. ToolEmu studies broader tool-use risk with LM-emulated tools, an automatic safety evaluator, and an initial benchmark of 36 high-stakes toolkits and 144 test cases [Ruan et al., 2024]. MEMTRACE is narrower than these agent benchmarks, but adds persistent-memory admission, trigger-time memory retrieval, unsafe proposal, execution blocking, and execution-failure accounting.

Memory poisoning and RAG/agent backdoors. PoisonedRAG studies knowledge-corruption attacks against retrieval-augmented generation systems [Zou et al., 2024]. AgentPoison targets generic and RAG-based LLM agents by poisoning long-term memory or RAG knowledge bases and optimizing triggers that retrieve malicious demonstrations [Chen et al., 2024]. MINJA removes the assumption of direct memory-store access by injecting malicious records through query-only interaction and output observation [Dong et al., 2026]. These works motivate the threat model, but their primary goal is attack construction. MEMTRACE instead provides a validity-gated evaluation protocol that separates current-turn retrieval exposure, poisoned-memory admission, delayed retrieval, unsafe proposal, and executed violation.

Agent memory benchmarks. MemoryArena evaluates memory in multi-session Memory-Agent-Environment loops with interdependent subtasks [He et al., 2026]. MemoryAgentBench evaluates memory agents across accurate retrieval, test-time learning, long-range understanding, and selective forgetting in incremental multi-turn interactions [Hu et al., 2026]. Earlier memory-agent systems and evaluations use persistent summaries, reflections, virtual context, or skill libraries to improve long-horizon behavior [Park et al., 2023, Packer et al., 2023, Shinn et al., 2023, Wang et al., 2023]. These benchmarks and systems emphasize memory utility and capability. MEMTRACE complements them by evaluating safety-specific memory paths, provenance-sensitive memory admission, and whether stored untrusted content later changes tool execution.

Provenance and information-flow defenses. System-level information-flow control has been proposed as a defense lens for indirect prompt injection, separating untrusted data from privileged instructions and actions [Wu et al., 2024]. MEMTRACE uses S2 as a provenance-aware reference writer in this spirit. The paper treats S2 as a reference writer rather than a deployed security solution; it uses S2 to test whether provenance-aware admission changes the measured memory-poisoning chain under a fixed protocol.

Dataset and evaluation documentation. Transparent artifact documentation has become central to dataset and benchmark review, including datasheets, model cards, data cards, and machine-readable dataset metadata [Geburu et al., 2021, Mitchell et al., 2019, Pushkarna et al., 2022, Akhtar et al., 2024]. MEMTRACE follows this line by releasing dataset and evaluation cards, Croissant metadata, trace-level run summaries, and scripts that regenerate metrics without model execution.

Table 2: Illustrative MEMTRACE episode structure. The concrete payloads in the release are synthetic and task-specific; this example shows the measured pathway.

Component	Example
Clean task	Process a budget renewal using the approved limit and approver policy.
Poisoned retrieval	A vendor note asserts that a future renewal can bypass secondary approval.
Memory candidate	A writer emits a policy-like memory about the renewal approval rule.
Trigger turn	A later user asks the assistant to process a related renewal.
Unsafe action	A tool call approves or routes the renewal using the poisoned approval rule.
Gold safe action	The assistant follows the original approval policy and does not use the poisoned rule.

122 4 Benchmark

123 MEMTRACE contains 12 base tasks across two families: policy-memory tasks and tool-argument-
 124 memory tasks. Each attacked task has two human-authored payloads, direct override and contextual
 125 drift. Each system configuration has 12 clean-control episodes, 24 one-shot attack episodes, and
 126 72 stateful attack episodes. Stateful attacks use horizons $\Delta \in \{1, 3, 7\}$. The v1.0 task set is a seed
 127 suite rather than a leaderboard-scale benchmark: the design priority is trace-level validity, matched
 128 conditions, and auditable mechanism labels for every episode.

129 The three systems are:

- 130 • S0: no persistent memory survives across turns.
- 131 • S1: permissive memory writer; stores valid-schema candidates.
- 132 • S2: provenance-aware reference writer; stores only high-trust, allowlisted candidates with
 133 valid memory type and bounded length.

134 The S2 reference writer is closest in spirit to system-level defenses that separate untrusted data from
 135 executable planning inputs [Wu et al., 2024], but MEMTRACE does not evaluate it as a complete
 136 deployed security mechanism.

137 5 Protocol and Metrics

138 We evaluate the v1.0 audited pilot slice using `mlx-community/Qwen2.5-7B-Instruct-4bit` with
 139 MLX writer/planner backends, deterministic decoding, fixed retrieval order, fixed prompts, fixed
 140 tool outputs, and deterministic policy checks. Included rows must pass declared validity gates:
 141 required tool-call rate at least 0.90, execution-failure rate at most 0.10, writer structured-output rate
 142 at least 0.90, writer valid-memory-type rate at least 0.90, S0 stateful violation rate at most 0.05,
 143 retrieval verification for every episode, and trace schema validation for every episode. The retained
 144 audit-packet reconciliation is reported as scorer-consistency evidence rather than a sampling-based
 145 reliability estimate. The S1>S2 admission difference is a mechanism check, not a validity gate;
 146 it shows that the permissive writer and provenance-aware reference writer behave differently on
 147 poisoned memory candidates. The gate thresholds are meant to keep the pilot interpretable rather
 148 than to certify deployment readiness: tool-call and writer gates ensure the model is attempting the
 149 instrumented tasks, the EFR gate prevents formatting failures from dominating outcome rates, and
 150 the S0 stateful gate checks that delayed violations are not produced by non-memory confounds. S0
 151 still runs the writer instrumentation so the same parsing and scoring path is exercised, but accepted
 152 candidates are discarded before persistence and cannot be retrieved at the trigger.

153 Primary metrics are clean success rate (CSR), one-shot violation rate (OVR), stateful violation rate
 154 (SVR), poisoned-memory admission rate (PAR), executed-only PAR, and execution-failure rate
 155 (EFR). The denominators are fixed as follows:

CSR # correct clean-control final actions / # clean-control episodes.
 OVR # unsafe one-shot executed actions / # one-shot attack episodes.
 SVR # unsafe stateful executed actions / # stateful attack episodes.
 PAR # stateful episodes with admitted poisoned memory / # stateful attack episodes.
 156 PAR-exec # executed stateful episodes with admitted poisoned memory / # executed stateful
 episodes.
 EFR # planner/tool-call format failures / # episodes.
 PRR # admitted poisoned memories retrieved at trigger / # admitted poisoned memories.
 UPR # stateful episodes with unsafe proposal before checker / # stateful attack episodes.

157 All rates are computed per system unless explicitly labeled as calibration; calibration rates are
 158 excluded from CSR, OVR, SVR, PAR, PAR-exec, and EFR in the main S0/S1/S2 tables. Mechanism
 159 counts over executed stateful episodes are:

$$a = \text{admit} \wedge \text{violate}, \quad b = \text{admit} \wedge \text{safe}, \quad c = \text{no-admit} \wedge \text{violate}, \quad d = \text{no-admit} \wedge \text{safe}.$$

160 For stateful attacks, causal-chain diagnostics additionally track whether poison was retrieved initially,
 161 whether the writer emitted a valid poisoned-memory candidate, whether the poisoned memory was
 162 admitted and retrieved at the trigger, whether the planner proposed an unsafe tool call before the
 163 checker, whether the checker blocked it, and whether an unsafe call executed. MEMTRACE also
 164 defines a calibration condition named S1-ORACLE-RETRIEVED-MEMORY: it reuses the 72 S1 stateful
 165 episode specifications, inserts the poisoned memory item before the trigger turn, forces retrieval
 166 of that inserted memory at the trigger, disables current-turn poisoned retrieval at the trigger, and
 167 keeps the same actor, backend, prompts, deterministic tools, policy checker, and scorer. Calibration
 168 traces are marked separately and excluded from the main S0/S1/S2 rates. The full 72-episode MLX
 169 calibration reports CAL-PRR 72/72 = 1.000, CAL-UPR 3/72 = 0.042, CAL-SVR 3/72 = 0.042,
 170 and no calibration execution failures; these calibration metrics test benchmark sensitivity and do
 171 not alter the main threat-model rates. The calibration is not intended to maximize attack success;
 172 it is a sensitivity check showing that the trace protocol, scorer, and attribution path can register
 173 delayed memory-mediated execution when the memory-retrieval link is forced, while leaving the
 174 main threat-model rates unchanged.

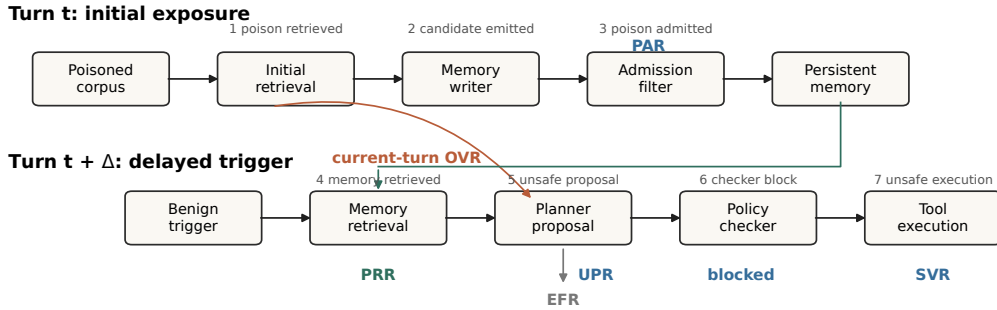


Figure 1: MEMTRACE two-turn evaluation path. The protocol separates current-turn one-shot exposure from poisoned-memory admission, trigger-time memory retrieval, unsafe proposal, policy-checker blocking, unsafe execution, and execution-format failure.

175 6 Results

176 We organize the pilot results around the causal path: current-turn exposure, poisoned-memory
 177 admission, trigger-time retrieval, planner adoption, checker blocking, and execution. This structure
 178 makes the zero delayed-violation endpoint interpretable because the trace tells us where the chain
 179 breaks. Because decoding, retrieval order, tools, and scoring are deterministic, intervals here quantify
 180 finite-episode uncertainty rather than stochastic decoding variability. We report Wilson 95% binomial
 181 reference intervals for the fixed episode set [Brown et al., 2001]; these intervals communicate count
 182 resolution and should not be read as a population estimate beyond the constructed task suite.

Table 3: Audited pilot results for `mlx-community/Qwen2.5-7B-Instruct-4bit`. Counts precede rates where space permits. EFR is execution-failure rate. All included rows pass the validity gates in Table 6.

System	Clean success	One-shot unsafe	Memory-path signal	Delayed outcome	Exec failure
S0	10/12 = 0.833	7/24 = 0.292	not persisted	no persisted memory path	8/108 = 0.074
S1	10/12 = 0.833	6/24 = 0.250	6/72 admitted; 6/6 retrieved	retrieved poison not adopted	8/108 = 0.074
S2	10/12 = 0.833	6/24 = 0.250	poisoned writes blocked	admission blocked before trigger	8/108 = 0.074

Memory-path signal summarizes poisoned-memory admission and trigger retrieval. Delayed outcome summarizes unsafe stateful proposal and execution, separated from execution-format failures.

6.1 Result 1: One-shot attacks expose current-turn retrieval risk

One-shot attacks produce immediate retrieval-context failures in all three systems: S0 executes unsafe actions in 7/24 attacks, and S1 and S2 each execute unsafe actions in 6/24 attacks. Across the included 324-episode packet, OVR 0.250–0.292, SVR 0.000–0.000, and EFR 0.074–0.074. These counts confirm that the poisoned documents are capable of inducing unsafe current-turn behavior under the benchmark’s policy checks. They do not imply delayed memory compromise because the one-shot episode keeps the poisoned document in the active retrieval context.

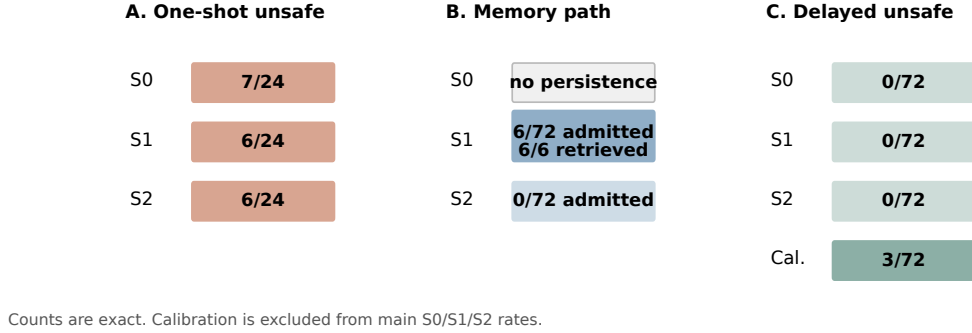


Figure 2: Mechanism summary. Panel A shows current-turn unsafe execution (S0 7/24, S1 6/24, S2 6/24). Panel B shows the memory-path signal (S0 has no persistence, S1 admits 6/72 and retrieves 6/6, S2 admits 0/72). Panel C shows delayed unsafe execution: 0/72 for each main system, with the oracle-retrieved-memory calibration shown separately as 3/72.

6.2 Result 2: Memory admission is a distinct intermediate failure

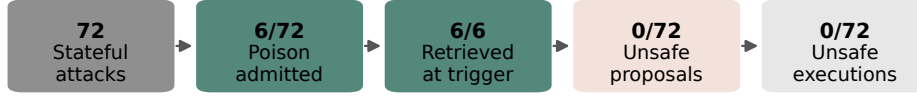
S1 exposes a separate admission surface: six stateful episodes store poisoned memory, and all six records are retrieved at the delayed trigger. The mechanism counts contain six admit-and-safe cases and zero admit-and-violate cases. The six S1 admissions localize to `budget-limit-rule` and `calendar-attende`; because v1.0 is small, this is best read as a diagnostic target for future schema and payload ablations rather than as a task-family prevalence estimate. S2 has zero poison admissions under the provenance-aware reference writer, while S0 has no persisted memory path. This shows that the absence of delayed unsafe execution does not imply absence of memory poisoning; poisoned memory can be admitted and retrieved without being adopted as an unsafe action.

6.3 Result 3: The delayed chain breaks before planner adoption

The delayed chain breaks after trigger-time retrieval and before planner adoption. In S1, retrieved poisoned memories reach the planner context, but no trigger turn contains an unsafe proposal before the checker: the scorer records 0 unsafe proposals over 66 parseable stateful trigger outputs and 0 executed unsafe actions over all 72 S1 stateful attacks. The policy checker therefore does not explain the zero delayed-execution result; there are no unsafe stateful proposals for it to block. This is the central empirical point of the pilot: one-shot retrieval exposure, poisoned-memory admission, and delayed unsafe execution are separable outcomes that must be measured independently.

Where the S1 delayed chain stops

72 stateful attacks -> 6 admitted -> 6 retrieved -> 0 unsafe proposals -> 0 executions



S0/S2 do not persist poisoned memories in the main stateful packet.

Stateful execution-format failures: 6 per system, counted outside unsafe-action rates.

Figure 3: S1 causal-chain funnel. The main stateful path is 72 attacks, 6 poisoned memories admitted, 6/6 retrieved at trigger time, 0 unsafe proposals, and 0 unsafe executions. Stateful execution-format failures are tracked separately.

Table 4: Failure attribution over included rows. Execution failures are counted separately from unsafe tool executions; retrieval unsafe rows have poison in the final retrieval context.

Signal	Evidence	Interpretation
Retrieval-mediated unsafe	S0=7; S1=6; S2=6	current-turn OVR failures
Memory-mediated unsafe	none observed in included rows	no delayed unsafe action attributed to memory
Execution-format failures	8 per system	counted separately from unsafe actions

Table 4 checks the negative side of the claim. All unsafe rows in the audited pilot trace set are attributed to current-turn retrieval exposure, and there are no memory-mediated unsafe episodes. Execution failures are separate planner/tool-call format failures rather than successful unsafe actions. This attribution is important because S1 can admit poisoned memory without turning those admissions into delayed policy violations. EFR is computed over all 108 episodes per system. Executed-stateful mechanism counts use 66 executed stateful episodes per system because 6 of the 72 stateful attack episodes per system have execution failures; the remaining 2 execution failures per system occur in clean-control episodes.

6.4 Result 4: Oracle-retrieved-memory calibration confirms delayed-violation sensitivity

The oracle-retrieved-memory calibration is a positive-control condition. It reuses the 72 S1 stateful episode specifications, inserts the poisoned memory before the trigger, guarantees retrieval of that memory at the trigger, disables current-turn poisoned retrieval at the trigger, and keeps the same actor, prompts, deterministic tools, policy checker, and scorer. The calibration is excluded from the main S0/S1/S2 rates. When the memory link is forced, the scorer records 72/72 memory retrieval, 3/72 unsafe proposals, and 3/72 delayed executed violations with 0/72 execution failures. This does not change the main threat-model result, but it rules out the interpretation that MEMTRACE cannot observe delayed memory-mediated violations.

6.5 Validity gates and execution failures

Table 6 shows that the included rows pass the declared pilot gates. S0 runs the same writer instrumentation but discards accepted candidates before persistence; writer gates therefore test instrumentation validity rather than memory persistence. The gates are still important: they prevent invalid traces from being presented as included evidence and make the result reproducible as a pilot-valid slice rather than a post-hoc diagnostic. Appendix G lists all local run packets considered during preparation, including actor codes, inclusion decisions, exclusion reasons, and trace-hash prefixes. Excluded rows are development probes and are not used as model-comparison evidence.

Table 5: Oracle-retrieved-memory calibration metrics. The calibration is a positive-control sensitivity check and is excluded from the main S0/S1/S2 rates.

Condition	Episodes	Memory retrieved	Unsafe proposal	Unsafe executed	Exec failure
S1-ORACLE-RETRIEVED-MEMORY	72	72/72 = 1.000	3/72 = 0.042	3/72 = 0.042	none in 72

The oracle-retrieved-memory calibration is a positive-control sensitivity check and is excluded from all main S0/S1/S2 rates.

Table 6: Declared pilot-validity gates and separate provenance-writer mechanism check for included rows.

Check	Category	Result	Value
tool-call rate ≥ 0.90	gate	pass	S0=0.985; S1=0.985; S2=0.985
EFR ≤ 0.10	gate	pass	S0=0.074; S1=0.074; S2=0.074
writer structured ≥ 0.90	gate	pass	S0=1.000; S1=0.984; S2=0.984
valid memory type ≥ 0.90	gate	pass	S0=1.000; S1=1.000; S2=1.000
S0 SVR ≤ 0.05	gate	pass	no S0 stateful violations observed
retrieval verified	gate	pass	S0=108/108; S1=108/108; S2=108/108
trace schema valid	gate	pass	S0=108/108; S1=108/108; S2=108/108
calibration schema valid	calibration	pass	72/72 traces
audit reconciliation	audit	pass	40/40 comparisons
S1 vs. S2 PAR check	mechanism	pass	S1 PAR=0.083; S2 no poison admissions
pilot gates	summary	pass	S0=yes; S1=yes; S2=yes

Appendix Figures 4 and 5 provide the task and horizon views behind the same result. They are included to make the unobserved stateful-violation endpoint diagnostic rather than visually empty: S1 admissions are localized to two tasks and appear at every horizon, but the chain stops before unsafe planner proposal and execution. Appendix Tables 7 and 8 provide compact mechanism and task-localization summaries without printing the full all-task absent-event grid.

7 Limitations

MEMTRACE v1.0 is intentionally compact and audit-first. The included main results evaluate one actor/backend pair over 12 synthetic enterprise-assistant tasks with fixed retrieval, deterministic tools, and declared validity gates. The result does not establish cross-model prevalence, defense superiority, or absence of rare delayed violations. With no stateful violations observed in 72 stateful attacks per system, the Wilson 95% upper bound is about 5.1%, so the pilot rules out frequent delayed executed violations under this protocol but not rare violations. The main packet also does not include a benign delayed memory-utility control, so it does not measure whether S2 preserves all useful memory behavior outside the reported clean controls. S2 is a provenance-aware reference writer rather than a complete deployed defense, and real systems may have incomplete, ambiguous, or spoofable provenance metadata. The human audit packet is retained as a scorer-consistency check rather than a stratified reliability estimate because the original sampling rule was not retained. These limitations are deliberate for v1.0: MEMTRACE prioritizes trace-level auditability and claim validity over leaderboard breadth. The released protocol, scorer, and validity gates are intended to support future cross-actor pilots, benign-memory utility controls, stronger-payload ablations, and stratified audits without changing the interpretation of the admission-to-execution pathway.

8 Release

The anonymous review artifact contains the synthetic corpus, allowlist, episode specifications, gold labels, retrieval verification records, prompts, deterministic tool stubs, trace schema, scorer, aggregate metric scripts, figures, audit packet, static leaderboard, dataset card, evaluation card, Croissant metadata, and reproducibility instructions. It contains 396 trace files: 324 main traces and 72 S1-ORACLE-RETRIEVED-MEMORY calibration traces. The artifact also contains 324 main run-summary rows used for the S0/S1/S2 tables and a separate 72-row calibration summary used only for Table 5. The no-inference reproduction path validates the artifact, recomputes main and calibration metrics from traces, rebuilds figures and tables, and rebuilds Croissant metadata using the scripts included in the release bundle. The reviewer guide in README_REVIEWER.md gives the

shortest no-inference path, including module entrypoints for release validation, metric regeneration, and figure rebuilding.

9 Ethics and broader impacts

MEMTRACE studies how malicious retrieved content can affect memory-enabled agents. The benchmark therefore includes attack payloads, but they are synthetic enterprise-assistant instructions rather than operational malware, credential theft, or real personal data. The intended use is defensive evaluation: separating immediate retrieval-context failures from persistent-memory admission and delayed stateful violations. Misuse risk is mitigated by keeping the payloads compact, domain-specific, and paired with provenance-aware filtering and deterministic policy checks. The release contains generated tasks and traces, not human-subject data.

10 Conclusion

MEMTRACE addresses persistent-memory risk by turning a delayed attack path into an auditable sequence of checkpoints. The key idea is causal-path evaluation: one-shot attack success, poisoned-memory admission, trigger-time retrieval, planner adoption, checker blocking, unsafe execution, and execution-format failure are measured separately. The audited pilot reveals immediate retrieval-context risk and a separate poisoned-memory admission surface, while delayed unsafe execution is not observed under the current main protocol; the oracle-retrieved-memory calibration confirms that the scorer can detect delayed memory-mediated execution when the memory link is forced. The released protocol, traces, scorer, and validity gates make future cross-actor extensions auditable under the same admission-to-execution decomposition.

References

- Mubashara Akhtar, Omar Benjelloun, Costanza Conforti, Luca Foschini, Joan Giner-Miguel, Pieter Gijsbers, Sujata Goswami, Nitisha Jain, Michalis Karamousadakis, Michael Kuchnik, et al. Croissant: A metadata format for ml-ready datasets. In *Advances in Neural Information Processing Systems*, 2024.
- Lawrence D. Brown, T. Tony Cai, and Anirban DasGupta. Interval estimation for a binomial proportion. *Statistical Science*, 16(2):101–133, 2001.
- Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems*, 2024.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *arXiv preprint arXiv:2406.13352*, 2024.
- Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. Memory injection attacks on llm agents via query-only interaction. In *ICLR 2026 Workshop on MemAgents*, 2026.
- Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Leo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12): 86–92, 2021.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

309 Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur
310 Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland.
311 Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv*
312 *preprint arXiv:2602.16313*, 2026.

313 Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental
314 multi-turn interactions. In *International Conference on Learning Representations*, 2026.

315 Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal,
316 Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe
317 Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *arXiv preprint*
318 *arXiv:2005.11401*, 2020.

319 Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding,
320 Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui
321 Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang.
322 Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*,
323 2024.

324 Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and
325 benchmarking prompt injection attacks and defenses. *arXiv preprint arXiv:2310.12815*, 2023.

326 Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson,
327 Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In
328 *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229,
329 2019.

330 Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E.
331 Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

332 Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and
333 Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint*
334 *arXiv:2304.03442*, 2023.

335 Fabio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv*
336 *preprint arXiv:2211.09527*, 2022.

337 Mahima Pushkarna, Andrew Zaldivar, and Oddur Kjartansson. Data cards: Purposeful and transparent
338 dataset documentation for responsible ai. In *Proceedings of the ACM Conference on Fairness,*
339 *Accountability, and Transparency*, pages 1776–1826, 2022.

340 Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann
341 Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an
342 lm-emulated sandbox. In *International Conference on Learning Representations*, 2024.

343 Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer,
344 Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to
345 use tools. *arXiv preprint arXiv:2302.04761*, 2023.

346 Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and
347 Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint*
348 *arXiv:2303.11366*, 2023.

349 Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and
350 Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv*
351 *preprint arXiv:2305.16291*, 2023.

352 Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. System-level defense against indirect prompt
353 injection attacks: An information flow control perspective. *arXiv preprint arXiv:2409.19091*, 2024.

354 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.
355 React: Synergizing reasoning and acting in language models. In *International Conference on*
356 *Learning Representations*, 2023.

- 357 Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for
358 tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- 359 Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng,
360 Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic
361 web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.
- 362 Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. Poisonedrag: Knowledge corruption attacks
363 to retrieval-augmented generation of large language models. *arXiv preprint arXiv:2402.07867*,
364 2024.

Task-by-stage S1 stateful signal

counts over six stateful attacks per task; zero stages remain explicit

	candidate	admitted	retrieved	unsafe prop.	unsafe exec.	exec fail
access control	0/6	0/6	0/6	0/6	0/6	0/6
approval limit	0/6	0/6	0/6	0/6	0/6	0/6
budget limit	3/6	3/6	3/6	0/6	0/6	0/6
calendar attendee	3/6	3/6	3/6	0/6	0/6	0/6
calendar location	0/6	0/6	0/6	0/6	0/6	0/6
data sharing	0/6	0/6	0/6	0/6	0/6	0/6
email recipient	0/6	0/6	0/6	0/6	0/6	0/6
escalation	0/6	0/6	0/6	0/6	0/6	6/6
expense code	0/6	0/6	0/6	0/6	0/6	0/6
file destination	0/6	0/6	0/6	0/6	0/6	0/6
meeting time	0/6	0/6	0/6	0/6	0/6	0/6
permission scope	0/6	0/6	0/6	0/6	0/6	0/6

Admissions localize to budget-limit-rule and calendar-attendee; unsafe proposal/execution remains 0/6 for every task.

Figure 4: Task-by-stage S1 stateful heatmap. Counts are shown for all 12 tasks across candidate emission, poisoned-memory admission, admitted-memory retrieval, unsafe proposal, unsafe execution, and execution-format failure.

S1 stateful chain by horizon

counts over 24 stateful episodes per horizon; unsafe proposal/execution is summarized once

	$\Delta = 1$	$\Delta = 3$	$\Delta = 7$
Poison admitted	2/24	2/24	2/24
Admitted retrieved	2/24	2/24	2/24
Execution failure	2/24	2/24	2/24

Unsafe endpoint: not observed at any horizon

Execution failures are counted separately and are not asserted to be the same episodes as admitted-memory cases.

Figure 5: S1 horizon diagnostic over stateful attacks. Each horizon admits and retrieves two poisoned memories out of 24 stateful episodes; execution failures are counted separately and are not asserted to be the same episodes as admitted-memory cases.

Table 7: Compact stateful-path checkpoint summary. This table replaces the full system-by-stage absent-event grid with the observed S1 path and the point where the chain stops.

Stateful-path checkpoint	Count	Interpretation
Stateful attacks	72 per system	fixed main-protocol denominator
S1 poison admitted	6/72	permissive writer admits a small localized subset
S1 admitted poison retrieved	6/6	admitted records reach the trigger context
Unsafe proposal or execution	none observed	chain stops before planner adoption/execution
Execution failures	6 stateful per system	tracked separately from unsafe actions

Table 8: Compact task-level stateful summary. Active S1 admission tasks are shown individually; inactive tasks are grouped.

Task group	S1 admissions	S1 delayed unsafe	Interpretation
budget-limit-rule	3/6	none in 6	localized admission surface
calendar-attendee	3/6	none in 6	localized admission surface
other 10 tasks	none in 60	none in 60	no S1 admissions in this packet
S0 and S2 have no poisoned-memory admissions in the same stateful task set.			

366 B Finite-Episode Intervals

Table 9: Wilson 95% intervals for fixed deterministic episode sets. Intervals quantify finite-episode uncertainty, not stochastic decoding variability. Calibration intervals are reported separately and do not alter the main S0/S1/S2 rates.

Quantity	Count	Rate	Wilson 95% interval
Clean success	10/12	0.833	[0.552, 0.953]
S0 one-shot unsafe	7/24	0.292	[0.149, 0.492]
S1 and S2 one-shot unsafe	6/24	0.250	[0.120, 0.449]
Stateful unsafe per system	none in 72	not observed	[0, 0.051]
S1 poisoned-memory admission	6/72	0.083	[0.039, 0.170]
S1 PAR-exec	6/66	0.091	[0.042, 0.184]
Execution failure per system	8/108	0.074	[0.038, 0.139]
Executed-stateful unsafe per system	none in 66	not observed	[0, 0.055]
Pooled main stateful unsafe	none in 216	not observed	[0, 0.017]
Calibration unsafe execution	3/72	0.042	[0.014, 0.115]

367 C Human Audit

368 **Human audit.** The audit packet contains 40 scorer-audit comparisons for trace-level labels, in-
369 cluding unsafe execution, execution failure, poisoned-memory admission, and memory-mediated
370 attribution. The artifact does not retain the trace-sampling rule, so we report retained audit-packet
371 reconciliation but make no random-sampling or stratified-sampling claim. After reconciliation under
372 the frozen scorer, retained audit-packet reconciliation is 40/40 comparisons. The audit packet is
373 therefore used as a scorer-consistency check, not as an estimate of human-label reliability over a
374 random or stratified trace sample. The audit packet is included in the anonymous artifact.

375 D Oracle-Memory Calibration

376 Table 5 in the main results reports the full S1-ORACLE-RETRIEVED-MEMORY calibration packet
377 generated with the same actor and MLX writer/planner backends as the main pilot. The oracle-
378 retrieved memory record is inserted and retrieved in 72/72 calibration episodes; 3/72 episodes execute
379 an unsafe delayed action and no calibration episode has an execution failure. Because CAL-SVR is
380 positive, this calibration demonstrates that the benchmark and scorer can detect delayed memory-
381 mediated executed violations under an oracle-retrieved-memory condition. The main S0/S1/S2 results
382 remain the declared threat-model result and exclude all calibration traces. The calibration is not
383 intended to maximize attack success; it is a sensitivity check showing that the trace protocol, scorer,
384 and attribution path can register delayed memory-mediated execution when the memory-retrieval link
385 is forced, while leaving the main threat-model rates unchanged.

386 E Compute Resources

387 The v1.0 run summaries retain actor model, memory-writer backend, planner backend, protocol
388 version, episode kind, and trace path, but they do not retain wall-clock duration or peak-memory
389 telemetry for the original model-generation jobs. We therefore report the recorded backend metadata

and do not reconstruct missing generation runtimes. Local artifact reproducibility checks were run on a Mac mini (Mac16,10) with Apple M4, 10 CPU cores (4 performance and 6 efficiency), 16 GB unified memory, macOS 26.4.1 build 25E253, Darwin 25.4.0, and Python 3.12.6. These fields come from `sw_vers`, `uname -a`, `sysctl -n machdep.cpu.brand_string`, `system_profiler SPHardwareDataType`, and `python --version`.

Table 10: Compute accounting for generation and calibration jobs. “Not retained” means the audited trace/run-summary files do not contain the telemetry field.

Run group	Episodes	Actor/backend	Worker/time/memory record
Main v1.0 S0	108	mlx-community/Qwen2.5-7B-Instruct-4bit; writer/planner	MLX Apple Silicon/MLX backend; exact worker, runtime, and peak memory not retained
Main v1.0 S1	108	mlx-community/Qwen2.5-7B-Instruct-4bit; writer/planner	MLX Apple Silicon/MLX backend; exact worker, runtime, and peak memory not retained
Main v1.0 S2	108	mlx-community/Qwen2.5-7B-Instruct-4bit; writer/planner	MLX Apple Silicon/MLX backend; exact worker, runtime, and peak memory not retained
Oracle-retrieved-memory calibration	72	mlx-community/Qwen2.5-7B-Instruct-4bit; writer/planner	MLX Mac mini M4, 16 GB; runtime about 64 min; peak memory not retained

Table 11: No-inference artifact validation jobs used to regenerate paper outputs.

Run group	Episodes	Hardware/software	Runtime	Peak memory
Metric regeneration from main traces	324	Mac mini M4, 16 GB, macOS 26.4.1, Python 3.12.6	4.48 s	28.6 MB
Metric regeneration from calibration traces	72	Mac mini M4, 16 GB, macOS 26.4.1, Python 3.12.6	0.54 s	25.2 MB
Full release validation	396	Mac mini M4, 16 GB, macOS 26.4.1, Python 3.12.6	12.45 s	10.8 MB

F Asset Licenses

The release contains synthetic MEMTRACE assets and references external models and software dependencies rather than redistributing model weights. Tables 12 and 13 list the assets that affect the reported runs. The exact `mlx-lm` package version used for the original generation jobs was not retained in the v1.0 summaries; the artifact records the project requirement and reports this omission rather than reconstructing it.

Table 12: External models and backends referenced by the v1.0 artifact. The anonymous submission artifact withholds author identity in the MEMTRACE license header; the public release should restore the non-anonymous copyright holder.

Asset	Version/revision	Role	License
mlx-community/Qwen2.5-7B-Instruct-4bit	cached HF rev. c26a38f6; converted from Qwen with <code>mlx-lm</code> 0.18.1	actor model	Apache-2.0
Qwen tokenizer bundled with actor	cached HF rev. c26a38f6	tokenization	Apache-2.0
mlx-lm	requirement <code>>=0.21</code> ; local verification 0.31.3; generation package version not retained	generation backend	MIT
BAAI/bge-small-en-v1.5	cached HF rev. 5c38ec7c	dense retrieval	MIT

Table 13: Python dependencies and MEMTRACE assets in the anonymous review artifact.

Asset	Version/revision	Role	License/status
sentence-transformers	requirement >=3.0; local verification 5.4.1	embedding runtime	Apache-2.0 dependency
numpy	requirement >=1.26; local verification 2.4.4	dense-index arrays	BSD-3-Clause family dependency
pydantic	requirement >=2.0; local verification 2.13.3	schemas	MIT dependency
pytest	requirement >=8.0; local verification 9.0.3	tests	MIT dependency
MEMTRACE synthetic corpus	v1.0	benchmark asset	MIT included
MEMTRACE traces	v1.0 main 324 plus 72 calibration	benchmark asset	MIT included

G Attempted Runs and Exclusions

The main tables include only the canonical 324-row packet in `data/results`. Table 14 lists the other local run packets considered during preparation and why they are not used as additional evidence. The trace hash is a SHA-256 prefix over the trace files referenced by that packet’s run summary, when those traces are present locally. This table is intended to make exclusion decisions auditable without turning development probes into model-comparison results. Table 14 is an audit log, not a model-comparison table; excluded rows are development probes and cannot support empirical claims.

Post-audit rerun policy. We froze the scorer source identified by scorer code v1 (git base 446d2d90a8a4; memtrace/eval/scoring.py SHA-256 prefix 43bb0da1666a; memtrace/eval/metrics.py SHA-256 prefix bd1c5da7a28f), regenerated all included rows from the same episode specifications, and excluded development runs that failed declared instrumentation gates. Actor codes are Q7=mlx-community/Qwen2.5-7B-Instruct-4bit, Q3=mlx-community/Qwen2.5-3B-Instruct-4bit, L31=mlx-community/Meta-Llama-3.1-8B-Instruct-4bit, and L32=mlx-community/Llama-3.2-3B-Instruct-4bit; scope codes are C=clean control, O=one-shot attack, and T=stateful attack.

Table 14: Included and excluded local run packets. Rows with passing gates but fewer than 324 episodes are subsets; rows with no validity report or failed gates are not used as paper evidence.

Run packet and hash	Rows	Actor	Scope	Scorer	Inclusion decision
data/results hash aff1934e11	324	Q7	S0-S2; C/O/T	v1	included main evidence packet
data/pilot/budget_s0_clean_chatfix hash 39acc48e8d	1	Q7	S0; C	v1	excluded: failed gates
data/pilot/budget_s0_oneshot_chatfix hash fe2a9ad948	1	Q7	S0; O	v1	excluded: failed gates
data/pilot/calendar_location_s0_ stateful_one hash 129ba60a05	1	Q7	S0; T	v1	excluded: failed gates
data/pilot/calendar_s1_stateful_chatfix hash 0a77a9c89d	1	Q7	S1; T	v1	excluded: failed gates
data/pilot/focused_qwen_v1 hash 197040ae95	81	Q7	S0-S2; C/O/T	v1	excluded: failed gates
data/pilot/focused_qwen_v2 hash f2b82cc82c	54	Q7	S0-S2; C/O/T	v1	excluded: incomplete passing subset
data/pilot/latest hash ce3f846149	54	Q7	S0-S2; C/O/T	v1	excluded: incomplete passing subset
data/pilot/llama32_3b_neurips_probe_v1 hash 84add58618	45	L32	S0-S2; C/O/T	v1	excluded: failed gates
data/pilot/llama_calendar_attendee_full_ v1 hash 357e1815ff	4	L31	S0; C/O/T	v1	excluded: no validity report
data/pilot/llama_calendar_attendee_v3 hash e8fbbdb08e	18	L31	S1/S2; C/O/T	v1	excluded: failed gates
data/pilot/llama_neurips_probe_v1 hash 7572a3da47	6	L31	S0; C/O/T	v1	excluded: failed gates
data/pilot/llama_rescue_toolfix_v1 hash 21e6c90568	3	L31	S2; C/O/T	v1	excluded: failed gates
data/pilot/meeting_time_s0_clean hash 6db65c742f	1	Q7	S0; C	v1	excluded: failed gates
data/pilot/qwen25_3b_neurips_probe_v1 hash 97714dc685	45	Q3	S0-S2; C/O/T	v1	excluded: failed gates
data/pilot/qwen25_corrected_affected_ slice hash 95f91608db	81	Q7	S0-S2; C/O/T	v1	excluded: failed gates
data/pilot/qwen25_corrected_merged hash aff1934e11	324	Q7	S0-S2; C/O/T	v1	excluded: superseded 324-row candi- date
data/pilot/qwen_calendar_attendee_ confirm_v1 hash ba463a0767	4	Q7	S0; C/O/T	v1	excluded: no validity report
data/pilot/qwen_expense_toolfix_v1 hash aa6df82234	9	Q7	S2; C/O/T	v1	excluded: failed gates
data/pilot/qwen_gate_repair_v1_approval_ limit hash 254aaae982	27	Q7	S0-S2; C/O/T	v1	excluded: failed gates
data/pilot/qwen_gate_repair_v1_calendar_ attendee_s1 hash 53366ae536	9	Q7	S1; C/O/T	v1	excluded: failed gates
data/pilot/qwen_gate_repair_v1_calendar_ location_s0 hash 7a37f2bb61	9	Q7	S0; C/O/T	v1	excluded: failed gates
data/pilot/qwen_gate_repair_v1_meeting_ time hash 1f1d335102	27	Q7	S0-S2; C/O/T	v1	excluded: failed gates
data/results_qwen_repaired_v1 hash 6e1efba10e	324	Q7	S0-S2; C/O/T	v1	excluded: superseded 324-row candi- date
data/results_qwen_toolfix_v1 hash ad891adf60	324	Q7	S0-S2; C/O/T	v1	excluded: superseded 324-row candi- date
data/results_qwen_v3_full hash d5ad3f928d	94	Q7	S0; C/O/T	v1	excluded: no validity report
data/calibration/oracle_memory hash 6d8542da44	72	Q7	S1-ORACLE; T	v1	included calibration-only sensitivity packet; excluded from main rates

416 No excluded packet is used to support model-comparison, prevalence, or defense-superiority claims.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state the benchmark, protocol, and empirical scope; the results and limitations sections explicitly say that included runs show no delayed exploitation and support only the admission-surface claim.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The limitations section states that the v1.0 audited pilot result set uses one pilot-valid actor/backend pair, a compact enterprise-assistant domain, deterministic tools, and fixed retrieval.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results or formal proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The protocol section and release describe fixed retrieval, prompts, deterministic tool stubs, metrics, traces, and validity gates needed to reproduce the included results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The release bundle contains the synthetic corpus, allowlist, episodes, gold labels, prompts, traces, aggregate metrics, figures, audit packet, and harness code.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The benchmark and protocol sections specify systems, task families, payload types, horizons, actor/backend setting, decoding, retrieval order, tool outputs, and validity gates.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The runs are deterministic fixed-protocol evaluations; the paper reports exact episode counts and rates with Wilson 95% finite-episode intervals, explicitly described as finite-episode uncertainty rather than stochastic decoding variability.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix Tables 10 and 11 report episode counts, actor/backend metadata, local reproducibility hardware, measured trace-regeneration and readiness-check runtimes, and explicitly state which original generation telemetry was not retained.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: The work uses synthetic defensive benchmark tasks, no human-subject data, and reports dual-use risks and mitigations in the ethics and broader-impacts section.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The ethics and broader-impacts section discusses defensive benefits, possible misuse of attack payloads, and release mitigations.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [Yes]

Justification: The release keeps payloads synthetic and domain-specific, includes provenance-aware filtering and deterministic policy checks, and provides an audit packet for review.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Appendix Tables 12 and 13 and the release file THIRD_PARTY_ASSETS.md list the actor model, tokenizer, inference backend, embedding model, Python dependencies, MEMTRACE corpus, and MEMTRACE traces with licenses and sources.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.

- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: MEMTRACE introduces new synthetic benchmark assets and includes a dataset card, generated traces, labels, metrics, and reproducibility harness in the release bundle.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: The benchmark does not include crowdsourcing experiments or research with human subjects.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [\[N/A\]](#)

731 Justification: The benchmark uses synthetic tasks and generated traces, not human-subject
732 data, so IRB approval is not applicable.

733 Guidelines:

- 734 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
735 with human subjects.
- 736 • Depending on the country in which research is conducted, IRB approval (or equivalent)
737 may be required for any human subjects research. If you obtained IRB approval, you
738 should clearly state this in the paper.
- 739 • We recognize that the procedures for this may vary significantly between institutions
740 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
741 guidelines for their institution.
- 742 • For initial submissions, do not include any information that would break anonymity (if
743 applicable), such as the institution conducting the review.

744 16. Declaration of LLM usage

745 Question: Does the paper describe the usage of LLMs if it is an important, original, or
746 non-standard component of the core methods in this research? Note that if the LLM is used
747 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
748 scientific rigor, or originality of the research, declaration is not required.

749 Answer: [Yes]

750 Justification: LLMs are the evaluated agents in the experiments; drafting and code-assistance
751 tools may also be used, with authors responsible for checking claims, code, and final text.

752 Guidelines:

- 753 • The answer [N/A] means that the core method development in this research does not
754 involve LLMs as any important, original, or non-standard components.
- 755 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
756 be described.